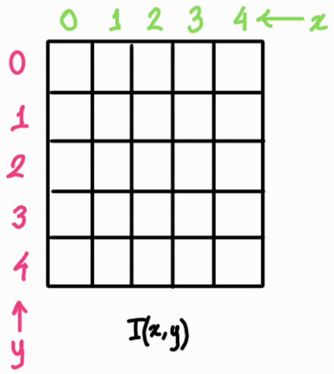
2D signals and convolution —

The most intuitive 2D signal is an image. We will consider a grayscale image (single channel).



I(x,y)

0	1	2	3	4
5	6	7	8	9
10	11	12	13	14
15	16	17	18	19
20	21	22	23	24

Linear indices

Each kernel for a 2D signal is also 2D (generally)

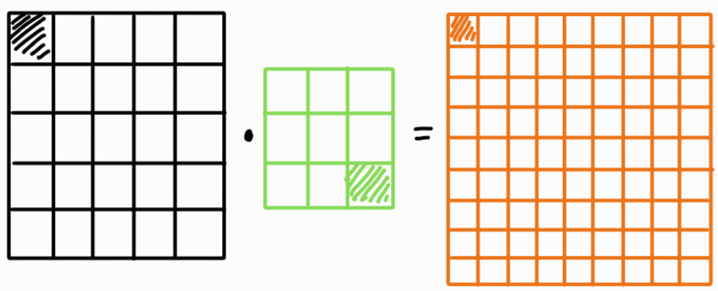
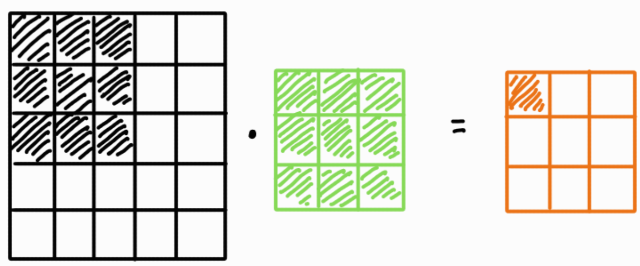
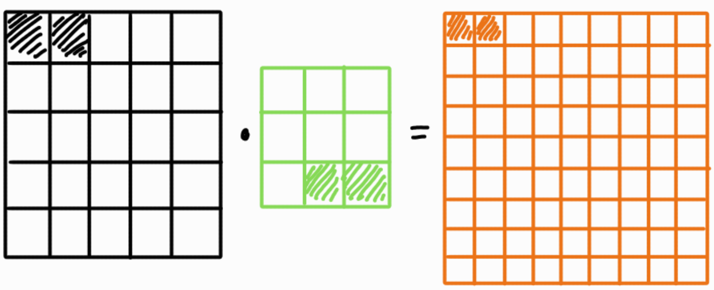
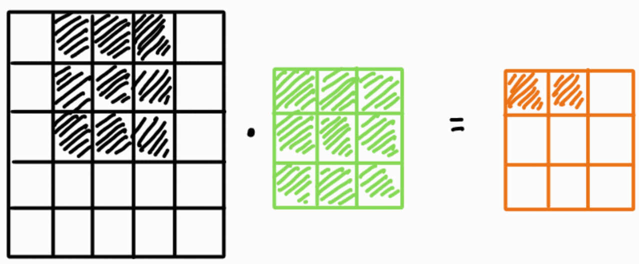
k_1	k_2	k_3
k_4	k_5	k_6
k_7	k_8	k_9

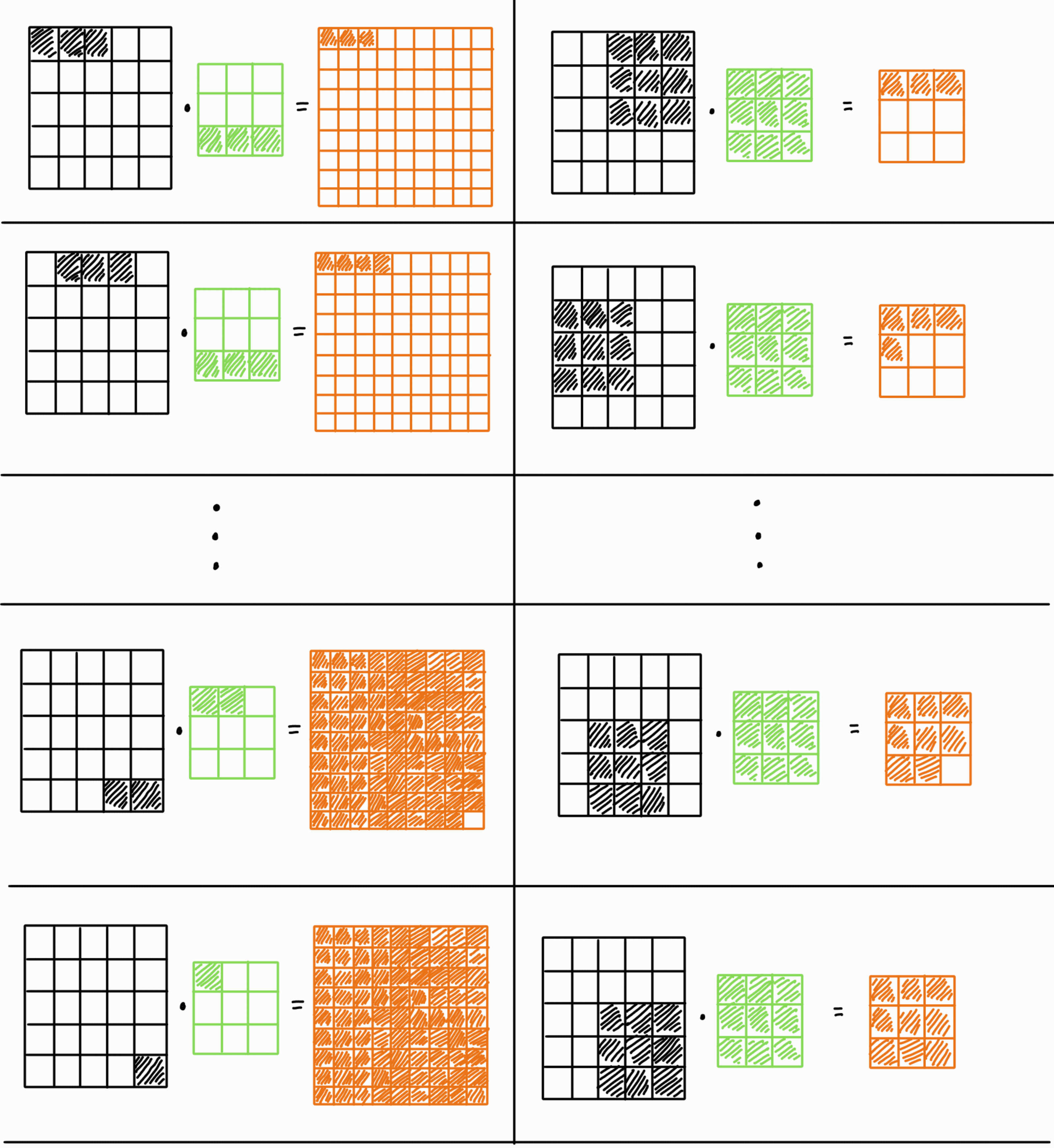
$k(x,y)$

k_9	k_8	k_7
k_6	k_5	k_4
k_3	k_2	k_1

Reversed kernel, $k'(x,y)$

Sliding path for 2D convolution —

General convolution	Truncated convolution
	
	



The kernel must be reversed at first.

For CNNs, we use the truncated variant.

2D filtering —

Original

LPF

HPF

Simple 2D LPF = $\frac{1}{9}\begin{bmatrix}1 & 1 & 1\\ 1 & 1 & 1\\ 1 & 1 & 1\end{bmatrix}$

Simple 2D HPF = $\begin{bmatrix}1 & 0 & -1\\ 1 & 0 & -1\\ 1 & 0 & -1\end{bmatrix}$

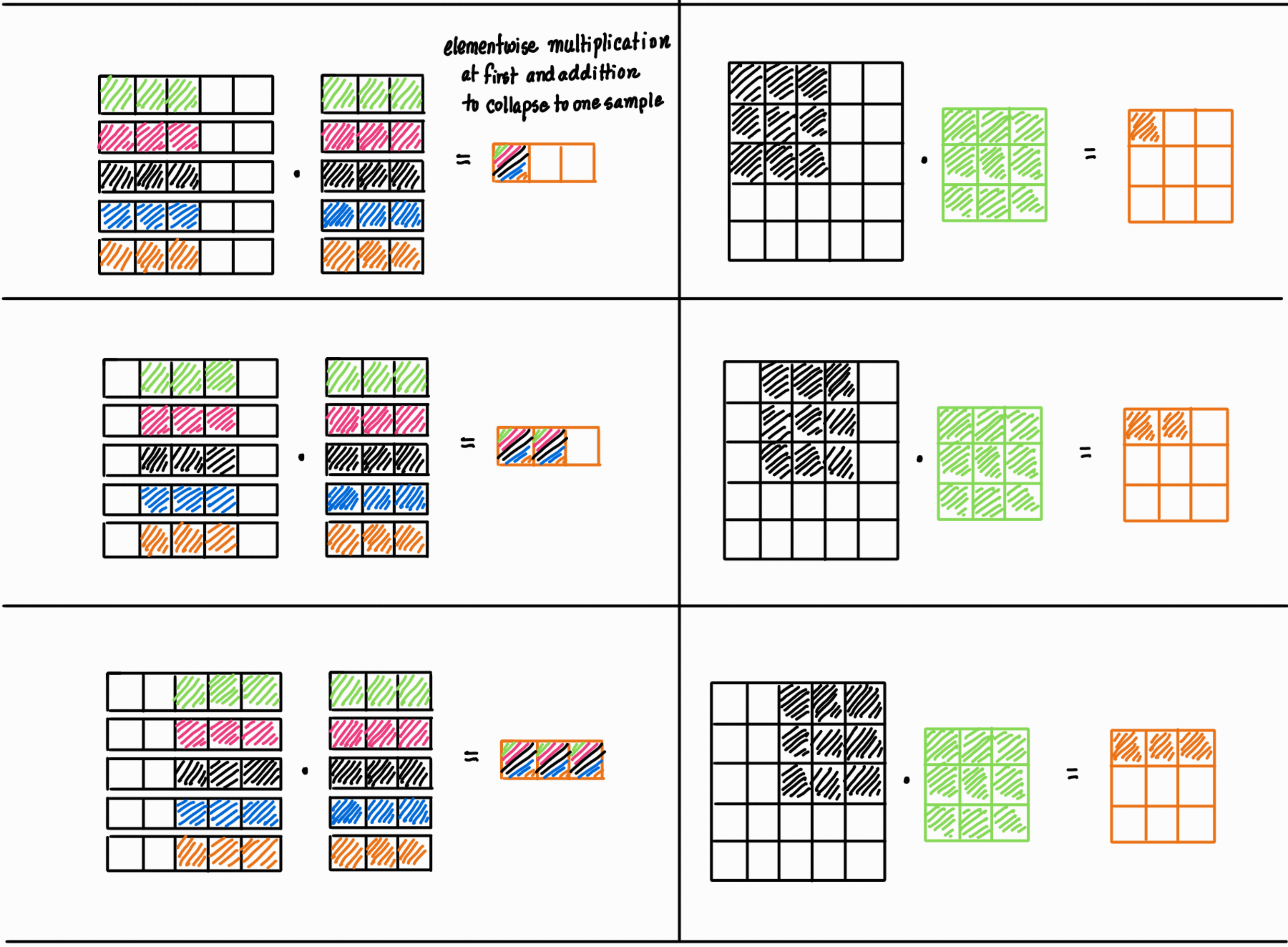
Assume you have an image, $I = \begin{bmatrix}2 & 64 & 23 & 13 & 125\\ 210 & 10 & 76 & 25 & 9\\ 10 & 186 & 14 & 53 & 31\\ 27 & 7 & 171 & 8 & 53\\ 221 & 29 & 9 & 251 & 7\end{bmatrix}$

Then find $I * HPF = \begin{bmatrix}2 & 64 & 23 & 13 & 125\\ 210 & 10 & 76 & 25 & 9\\ 10 & 186 & 14 & 53 & 31\\ 27 & 7 & 171 & 8 & 53\\ 221 & 29 & 9 & 251 & 7\end{bmatrix} \star \underbrace{\begin{bmatrix}-1 & 0 & 1\\ -1 & 0 & 1\\ -1 & 0 & 1\end{bmatrix}}_{\text{Reversed kernel}} = \begin{bmatrix}-109 & -169 & 52\\ 14 & -117 & -168\\ -64 & 90 & -103\end{bmatrix}$

1D multichannel filtering —

We did introduce multichannel signals in the 1st lecture. In most practical applications, your signals will be multichannelled. While you might be inclined to deal with 1D multichannelled data as 2D data, its not wise to do so. Lets see how filtering differs between 2 D data and multichannelled 1D data. We will stick to truncated convolution for this segment —

Multichannel 1D	Single channel 2D



All the channels have to be considered together for complete information unlike in single channel 2D. Each channel gets seperate kernel. A similar process is followed for multichannelled 2D data.

The bigger picture —

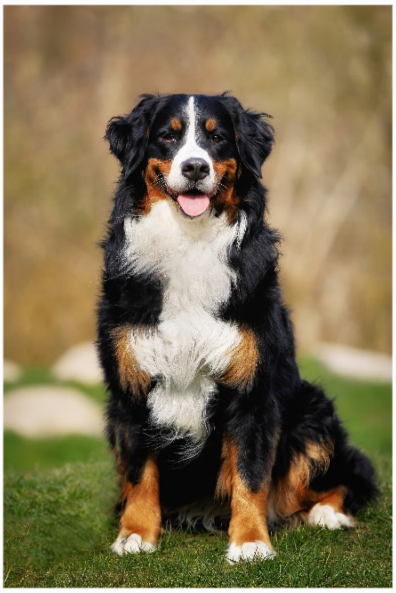
If you step back a bit, what are the filters doing? They are extracting some feature from the original signal. For example high pass filters highlights how the samples change. In images an HPF outputs the object edges. An LPF on the other hand outputs the overall trend in the signal. Complex filters can extract complex features from the signal.

So effectively, if we have carefully designed filters, we in theory should be able to isolate all features of the signal —

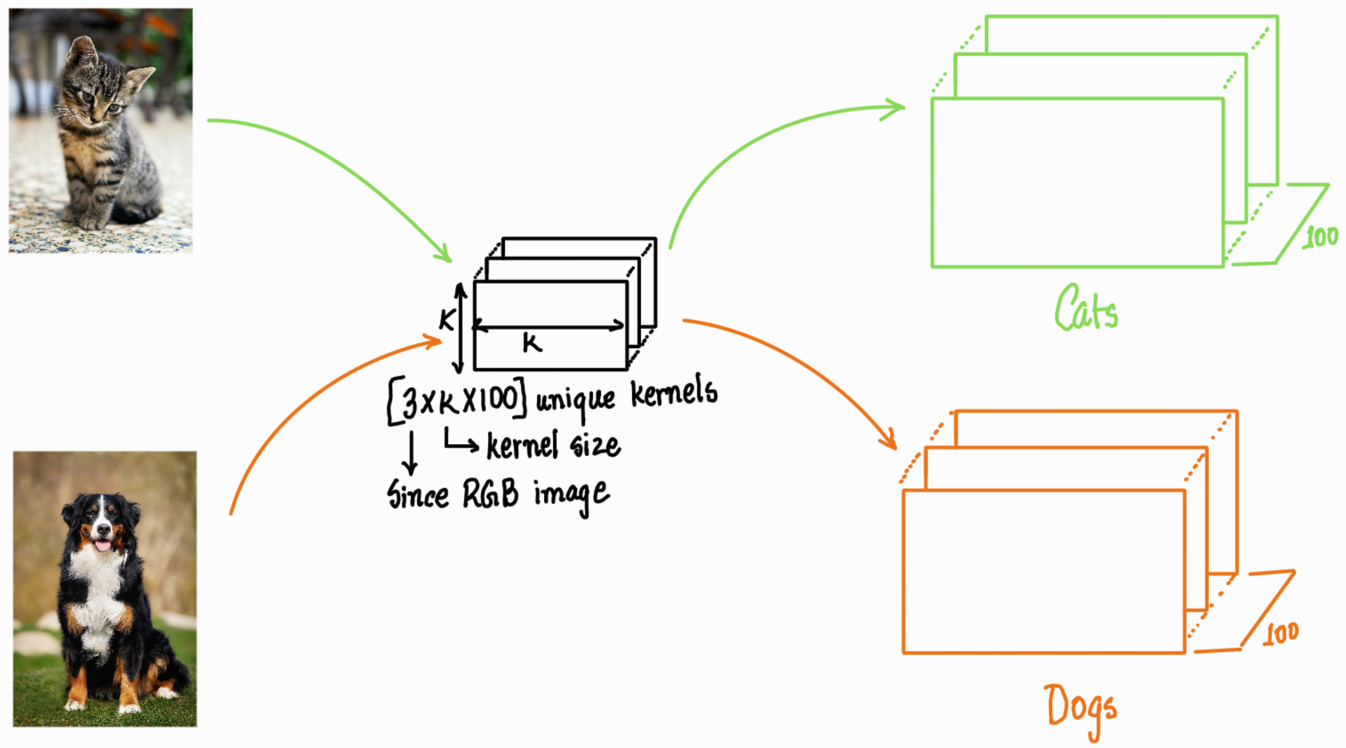
Signal → kernel 1 → feature 1
Signal → kernel 2 → feature 2
⋮
Signal → kernel n → feature n

What can we do with these features?

Lets say you have a cat and a dog pic —



Lets assume these images have a total of 100 distinguishing features that can be extracted using 100 different kernels from both pics ——



If these unique features can indeed distinguish a cat or a dog definitively, then we in theory, should be able to design a numeric algorithm that takes these features, do some processing and outputs cat or dog based on our features ——

$$\text{Image} \longrightarrow \boxed{\text{Convolutions}} \longrightarrow \text{Features} \longrightarrow \boxed{\text{Decision mechanism}} \longrightarrow \text{Cat/Dog}$$

But there are certain issues that needs to be tackled —

* We cannot design such task specific filters as such specific filters are unintuitive first of all and secondly most don't hold for all signals
* Even when we can design the filters, we cannot do so directly in time domain.
* Small kernels give bad filters. Large kernels are computationally heavy.
* Convolution based filters focus on the neighborhood, rather than the global outlook of the signal as kernel size is kept limited.

We solve the first 2 problems by not designing the filters ourselves. We let some iterative algorithm design these filters for us.

For the latter 2 problems, we simply cascade a bunch of filters to get better features.

We will learn more about it in the next lecture.